Machine Coding: Chips Input

Problem: Build a Chips (Tags) Input: the user types text and presses Enter or comma to turn it into a chip; chips can be removed.

Tech stack: React (Vite, JavaScript)

Time limit: 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify requirements**: what must work, what is out of scope.
2. **List interactions**: every user action and the expected result. This becomes your test checklist.
3. **Sketch the architecture**: components and who owns which state.
4. **Design the state**: the smallest state; derive the rest.
5. **Find the hard parts early**: tackle them before polishing UI.
6. **Build in layers**: simplest working version first, then add one working feature at a time.
7. **Cover states and edge cases**: empty, error, limits, keyboard, accessibility.
8. **Wrap up**: explain trade-offs and what you would add with more time.

Time budget (60 minutes)

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state	8 min
Core: render chips, add on Enter, remove on ×	15 min
Comma split, Backspace, paste, validation	15 min
Accessibility + styling	10 min
Edge-case pass + wrap-up	7 min

Tip: think out loud

Say every decision before you code it: "*I'll keep the chips in the parent so the form owns the value, and keep only the half-typed text inside the component.*" The interviewer grades your reasoning as much as the result. If you run out of time, a clear plan for the unfinished parts still earns points.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Type text; **Enter** or **comma** converts it into a chip.
- Each chip has a **× button** that removes it.
- **Backspace** in an empty input removes the last chip.
- **Paste** a list like `a, b, c` (or one per line) creates several chips at once.
- Trim whitespace; ignore empty values.
- Reject **duplicates** (case-insensitive) with a visible message.
- Enforce a **max number of chips** and a **max length** per chip.
- Show a counter like `3/8`.
- The parent can read the chips (controlled `value` / `onChange`).

Non-functional requirements

- **Performance:** chip lists are small (tens). A plain array is fine; no virtualization.
- **Scalability / reusability:** one `ChipsInput` component, configurable via props (`max`, `maxLength`, `placeholder`, `label`). Drop it into any form.
- **Error handling:** invalid input never crashes or silently disappears; the user sees why.
- **Accessibility:** proper `<label>`, chips in a list, remove buttons with `aria-label="Remove React"`, error announced with `aria-live`, fully keyboard-usable.
- **Usability:** clicking anywhere in the box focuses the input; Enter does not submit the parent form.
- **Extensibility:** easy to add suggestions/autocomplete or async validation later.

Clarifying questions to ask

1. Which keys create a chip: only Enter, or also comma / Tab / space?
2. Should duplicates be allowed? Case-sensitive or not?
3. Is there a max number of chips or max length per chip?
4. Should pasting a comma-separated list create multiple chips?
5. Should the component be controlled (parent owns the value) or self-contained?
6. Do we need autocomplete suggestions, or free text only?

Assumptions we will build with

- Enter and comma create chips; newline and tab also split pasted text.
- Duplicates are rejected, case-insensitive (`React` == `react`).
- Defaults: `max = 10` chips, `maxLength = 20` characters (configurable).
- Controlled component: parent passes `value` (array of strings) and `onChange`.
- Free text only, no autocomplete.

- Leaving the input (blur) commits whatever is typed, so nothing is lost on submit.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Type `react` + Enter	Chip "react" added, input cleared
Type `vue,`	Chip "vue" added on the comma, input cleared
Type `vue,nod`	"vue" becomes a chip, `nod` stays in the input
Enter on empty / whitespace input	Nothing happens
Type `go` + Enter	Chip "go" (trimmed)
Type duplicate `React` when `react` exists	Not added, error `"React" is already added`, text kept so user can edit
Add beyond max	Not added, error `You can add at most N chips`
Text longer than maxLength	Not added, error shown
Click × on a chip	Chip removed, focus back to input
Backspace on empty input	Last chip removed
Backspace with text	Normal character delete
Paste `a, b\nc`	Three chips added at once
Paste plain `hello`	Normal paste into the input
Start typing after an error	Error clears
Blur input with text	Text committed as chip
Enter inside a `<form>`	Chip added, form NOT submitted
Enter while IME is composing (Japanese/Chinese)	Confirms the word, no chip yet
Click empty area of the box	Input gets focus

---

3. Design the architecture

Module	Responsibility
`App`	Demo page; owns the chips array (like a form would)

Module	Responsibility
`ChipsInput`	Input text, keyboard/paste handling, error message, counter
`Chip`	Renders one chip with a remove button
`utils/chips.js`	Pure logic: split raw text into labels, validate and add labels

```
App  (chips state)
 └── ChipsInput  (text + error state, events)
      ├── Chip × n   (label, onRemove)
      └── <input>
utils/chips.js  ← used by ChipsInput (pure, testable without React)
```

Why split the logic into `utils`? Splitting, trimming, deduping and limits are pure functions. Pure functions are easy to unit test and easy to reason about. The component only wires events to that logic.

---

4. Design the state / data model

State or field	Owner	Purpose
`value` (string[])	Parent (`App`)	The chips. The actual form value
`text` (string)	`ChipsInput`	What the user is currently typing
`error` (string)	`ChipsInput`	Last validation message, `""` when fine

```
// Parent
const [skills, setSkills] = useState(["React", "JavaScript"]);

// Inside ChipsInput
const [text, setText] = useState("");
const [error, setError] = useState("");
```

Senior notes

- **Derived, not stored:** the counter (`value.length`), "is at max" (`value.length >= max`), placeholder visibility and `aria-invalid` (`Boolean(error)`) are all derived on render.
- **Ownership:** chips belong to the parent because a form needs to submit them. Typing text and errors are UI details, so they live inside the component.
- **Why strings, not objects with ids?** Duplicates are rejected, so each label is unique and can be the React `key`. If duplicates were allowed, we'd switch to `{ id, label }`.

---

5. Design the data structures and contracts

Example data

```
["React", "JavaScript", "Vue"]
```

Labels are unique (case-insensitive), which makes the label itself a stable key.

Component contract

```
<ChipsInput
  label="Skills"             // visible label, also used for aria
  value={skills}             // string[]  (controlled)
  onChange={setSkills}       // (next: string[]) => void
  max={8}                    // optional, default 10
  maxLength={20}             // optional, default 20
  placeholder="Type and press Enter"
/>
```

Pure helper contracts

```
splitLabels(" a, b,,\n c ")          // -> ["a", "b", "c"]

addLabels(["React"], ["react", "Go"], { max: 10, maxLength: 20 })
// -> { chips: ["React", "Go"], error: '"react" is already added' }
```

No API is needed. If labels had to be validated on a server, `addLabels` would become an async check before calling `onChange`.

---

6. Trace the data flow

The input text becomes `vue,nod` (a comma typed before `nod`, or a paste). Typing `vue` then `,` follows the same path with `rest = ""`:

```
<input> onChange("vue,nod")
   ↓
handleChange sees SEPARATOR in "vue,nod"
   ↓
parts = ["vue", "nod"]  -> rest = "nod", toCommit = "vue"
```

```
   ↓
commit("vue") -> splitLabels -> ["vue"]
   ↓
addLabels(value, ["vue"], { max, maxLength }) -> { chips: [..., "vue"], error: "" }
   ↓
onChange(chips)  -> parent setSkills -> re-render with new Chip
setError("")     setText("nod")
   ↓
Output: chip "vue" visible, input shows "nod", counter +1
```

---

7. Important technical challenges

Challenge 1: Splitting on comma while keeping the unfinished part

Problem: The user types fast: `react,vu`. If we only check `key === ","` in `onKeyDown`, pasted or autocorrected text with commas is missed. If we commit the whole string, `vu` becomes a chip too early.

Solution: Handle it in `onChange`: split by the separator, commit everything except the last part, keep the last part as the new input text.

```
const handleChange = (e) => {
  const next = e.target.value;
  if (!SEPARATOR.test(next)) { setText(next); setError(""); return; }
  const parts = next.split(SEPARATOR);
  const rest = parts.pop();      // unfinished text stays
  commit(parts.join(","));
  setText(rest);
};
```

Challenge 2: Validation (duplicates, limits) for one or many labels

Problem: A paste like `Go, go, Rust` must add `Go` and `Rust`, skip the duplicate `go`, and stop at the max. Checking only against the existing list misses duplicates inside the same paste.

Solution: A pure function with a `Set` of lowercased labels that grows as we add, so it also catches duplicates within the same batch. Return the reason for the last skip so the UI can show it.

```
export function addLabels(chips, labels, { max = Infinity, maxLength = Infinity } = {}) {
  const seen = new Set(chips.map((c) => c.toLowerCase()));
  const next = [...chips];
  let error = "";
  for (const label of labels) {
```

```
    if (next.length >= max) { error = `You can add at most ${max} chips`; break; }
    if (label.length > maxLength) { error = `"${label}" is longer than ${maxLength}
characters`; continue; }
    if (seen.has(label.toLowerCase())) { error = `"${label}" is already added`; continue; }
    seen.add(label.toLowerCase());
    next.push(label);
  }
  return { chips: next, error };
}
```

Challenge 3: Keyboard: Enter, Backspace and IME composition

Problem:

- Enter inside a `<form>` submits the form.
- A Japanese user presses Enter to confirm a word in the IME. Without care, that Enter creates a half-finished chip.
- Backspace on empty input should remove the last chip, but must not eat chips while the user is deleting characters.

Solution: `preventDefault` on Enter, skip when `e.nativeEvent.isComposing`, and only remove a chip when `text === ""`.

```
const handleKeyDown = (e) => {
  if (e.key === "Enter" && !e.nativeEvent.isComposing) {
    e.preventDefault();
    if (commit(text)) setText("");
  } else if (e.key === "Backspace" && text === "" && value.length > 0) {
    removeChip(value[value.length - 1]);
  }
};
```

Challenge 4: Pasting a multi-line list

Problem: A user copies a column from a spreadsheet (`Go\nRust\nTypeScript`). A single-line `<input>` strips the newlines, so the paste becomes `GoRustTypeScript`, one wrong chip.

Solution: Intercept `onPaste`, read the clipboard text, and if it has separators, split it ourselves. Plain text without separators is left to the browser.

```
const handlePaste = (e) => {
  const pasted = e.clipboardData.getData("text");
  if (!SEPARATOR.test(pasted)) return;
  e.preventDefault();
```

```
  if (commit(text + pasted)) setText("");
};
```

8. Implement in the correct order

#	Step	Working result
1	`App` with `useState` array, render chips as plain list	Static chips visible
2	`ChipsInput` with controlled `<input>` + Enter adds trimmed text	Can add chips
3	`Chip` component with × button	Can remove chips
4	Extract `splitLabels` / `addLabels` to `utils`; dedupe	No duplicates
5	Comma handling in `onChange` (keep the remainder)	Comma works
6	Backspace on empty input removes last chip	Keyboard delete
7	Max chips, max length, error message + counter	Limits enforced, user sees why
8	Paste handling for multi-value text	Bulk add
9	Blur commits, Enter `preventDefault`, IME check	Safe inside forms
10	Accessibility: label, list, aria-label, aria-live, focus on box click	Screen-reader friendly
11	Styling: wrapping flex box, focus ring, error border	Looks finished
12	Edge-case pass with the interaction table	Verified

9. Cover all states and edge cases

State / edge case	Handling
Initial (with default chips)	Chips rendered from `value` prop
Empty (no chips)	Placeholder shown, counter `0/max`
Typing	Text kept in local state; error cleared
Success (chip added)	Input cleared, chip appears, counter updates
Error: duplicate	Message shown, text kept for editing, red border
Error: too long	Message shown, chip not added
Error: max reached	Message shown, extra labels ignored

State / edge case	Handling
Whitespace-only input	Ignored
Leading/trailing spaces	Trimmed
`a,,b` / trailing comma	Empty parts filtered out
Duplicates inside one paste	Caught by the growing `Set`
Backspace with text	Normal delete, chips untouched
Backspace on empty, no chips	Nothing happens
Enter inside a `<form>`	`preventDefault`, no submit
IME composition	Enter ignored while composing
Blur with pending text	Committed (nothing lost on submit)
Long chip label	Blocked by `maxLength`; box wraps to new rows
Remove chip	Focus returns to input
Loading	Not applicable (no async work)

10. Final structure

```
_49_chips_input/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── App.jsx                 # demo, owns chips state
    ├── App.css                 # styles
    ├── components/
    │   ├── ChipsInput.jsx      # input, keyboard, paste, error, counter
    │   └── Chip.jsx            # one chip + remove button
    └── utils/
        └── chips.js            # splitLabels, addLabels (pure)
```

11. Full working code

src/utils/chips.js

```
// Comma, newline or tab ends a chip (typed or pasted)
export const SEPARATOR = /[,\n\t]/;

// "a, b,,  c " -> ["a", "b", "c"]
export const splitLabels = (raw) =>
  raw
    .split(SEPARATOR)
    .map((label) => label.trim())
    .filter(Boolean);

// Pure: returns the next chip list and why the last label was skipped (if any)
export function addLabels(chips, labels, { max = Infinity, maxLength = Infinity } = {}) {
  const seen = new Set(chips.map((chip) => chip.toLowerCase()));
  const next = [...chips];
  let error = "";

  for (const label of labels) {
    if (next.length >= max) {
      error = `You can add at most ${max} chips`;
      break;
    }
    if (label.length > maxLength) {
      error = `"${label}" is longer than ${maxLength} characters`;
      continue;
    }
    if (seen.has(label.toLowerCase())) {
      error = `"${label}" is already added`;
      continue;
    }
    seen.add(label.toLowerCase());
    next.push(label);
  }

  return { chips: next, error };
}
```

src/components/Chip.jsx

```
export default function Chip({ label, onRemove }) {
  return (
    <li className="chip">
      <span className="chip-label">{label}</span>
      <button
        type="button"
        className="chip-remove"
        aria-label={`Remove ${label}`}
        onClick={onRemove}
      >
        ×
      </button>
    </li>
  );
}
```

src/components/ChipsInput.jsx

```
import { useId, useRef, useState } from "react";
import Chip from "./Chip";
import { SEPARATOR, addLabels, splitLabels } from "../utils/chips";

export default function ChipsInput({
  label,
  value,
  onChange,
  max = 10,
  maxLength = 20,
  placeholder = "Type and press Enter",
}) {
  const [text, setText] = useState("");
  const [error, setError] = useState("");
  const inputRef = useRef(null);
  const inputId = useId();
  const errorId = useId();

  // Returns true when the raw text was consumed (added, or nothing to add)
  const commit = (raw) => {
    const labels = splitLabels(raw);
    if (labels.length === 0) return true;
```

```
  const result = addLabels(value, labels, { max, maxLength });
  const added = result.chips.length !== value.length;
  if (added) onChange(result.chips);
  setError(result.error);
  return added;
};

const removeChip = (chip) => {
  onChange(value.filter((c) => c !== chip));
  setError("");
  inputRef.current.focus();
};

const handleChange = (e) => {
  const next = e.target.value;
  if (!SEPARATOR.test(next)) {
    setText(next);
    setError("");
    return;
  }
  // "react,vu" -> add "react", keep "vu" in the input
  const parts = next.split(SEPARATOR);
  const rest = parts.pop();
  commit(parts.join(","));
  setText(rest);
};

const handleKeyDown = (e) => {
  // Enter while an IME (Chinese/Japanese) is composing confirms the word, not the chip
  if (e.key === "Enter" && !e.nativeEvent.isComposing) {
    e.preventDefault(); // don't submit the parent form
    if (commit(text)) setText("");
  } else if (e.key === "Backspace" && text === "" && value.length > 0) {
    removeChip(value[value.length - 1]);
  }
};

// Single-line inputs drop newlines on paste, so split the clipboard ourselves
const handlePaste = (e) => {
  const pasted = e.clipboardData.getData("text");
  if (!SEPARATOR.test(pasted)) return; // plain paste, let the browser do it
  e.preventDefault();
  // shortcut: ignores caret position (appends to current text), fine for tag inputs
  if (commit(text + pasted)) setText("");
```

```
  };

  const handleBlur = () => {
    if (commit(text)) setText("");
  };

  return (
    <div className="chips-field">
      <label htmlFor={inputId} className="chips-label">
        {label}
      </label>

      {/* Clicking anywhere in the box focuses the input */}
      <div
        className={`chips-box${error ? " has-error" : ""}`}
        onClick={() => inputRef.current.focus()}
      >
        <ul className="chips" aria-label={`${label} selected`}>
          {value.map((chip) => (
            <Chip key={chip} label={chip} onRemove={() => removeChip(chip)} />
          ))}
        </ul>
        <input
          id={inputId}
          ref={inputRef}
          className="chips-input"
          value={text}
          onChange={handleChange}
          onKeyDown={handleKeyDown}
          onPaste={handlePaste}
          onBlur={handleBlur}
          placeholder={value.length ? "" : placeholder}
          aria-invalid={Boolean(error)}
          aria-describedby={errorId}
        />
      </div>

      <div className="chips-meta">
        <p id={errorId} className="chips-error" aria-live="polite">
          {error}
        </p>
        <span className="chips-count">
          {value.length}/{max}
        </span>
```

```
      </div>
    </div>
  );
}
```

src/App.jsx

```
import { useState } from "react";
import ChipsInput from "./components/ChipsInput";
import "./App.css";

export default function App() {
  const [skills, setSkills] = useState(["React", "JavaScript"]);

  return (
    <main className="app">
      <h1>Chips Input</h1>
      <ChipsInput label="Skills" value={skills} onChange={setSkills} max={8} />
      <pre className="output">{JSON.stringify(skills)}</pre>
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

src/App.css

```
* {
  box-sizing: border-box;
}
```

```css
body {
  margin: 0;
  min-height: 100vh;
  font-family: system-ui, sans-serif;
  background: #f4f5f7;
  color: #1f2937;
  display: grid;
  place-items: center;
}

.app {
  width: min(520px, 100% - 32px);
}

.app h1 {
  font-size: 1.75rem;
}

.chips-label {
  display: block;
  font-weight: 600;
  margin-bottom: 6px;
}

.chips-box {
  display: flex;
  flex-wrap: wrap;
  gap: 6px;
  padding: 8px;
  min-height: 46px;
  background: #fff;
  border: 1px solid #cbd5e1;
  border-radius: 8px;
  cursor: text;
}

.chips-box:focus-within {
  border-color: #2563eb;
  box-shadow: 0 0 0 3px rgb(37 99 235 / 0.2);
}

.chips-box.has-error {
  border-color: #dc2626;
}
```

```
/* Chips and input share one wrapping row */
.chips {
  display: contents;
  list-style: none;
}

.chip {
  display: inline-flex;
  align-items: center;
  gap: 4px;
  padding: 2px 4px 2px 10px;
  background: #e0e7ff;
  color: #3730a3;
  border-radius: 999px;
  font-size: 0.9rem;
}

.chip-remove {
  border: 0;
  background: transparent;
  color: inherit;
  width: 22px;
  height: 22px;
  border-radius: 50%;
  cursor: pointer;
  font-size: 1rem;
  line-height: 1;
}

.chip-remove:hover,
.chip-remove:focus-visible {
  background: #c7d2fe;
}

.chips-input {
  flex: 1;
  min-width: 120px;
  border: 0;
  outline: 0;
  font: inherit;
  padding: 4px;
}
```

```
.chips-meta {
  display: flex;
  justify-content: space-between;
  font-size: 0.85rem;
  min-height: 1.5em;
  margin-top: 4px;
}

.chips-error {
  margin: 0;
  color: #dc2626;
}

.chips-count {
  color: #64748b;
}

.output {
  background: #1f2937;
  color: #e5e7eb;
  padding: 12px;
  border-radius: 8px;
  white-space: pre-wrap;
}
```

Quick self-check of the pure logic (node)

```
import assert from "node:assert/strict";
import { addLabels, splitLabels } from "./src/utils/chips.js";

assert.deepEqual(splitLabels(" a, b,,\n c\t "), ["a", "b", "c"]);
assert.deepEqual(splitLabels("   "), []);

assert.deepEqual(addLabels(["React"], ["Vue"]), { chips: ["React", "Vue"], error: "" });
assert.equal(addLabels(["React"], ["react"]).error, '"react" is already added');
assert.deepEqual(addLabels([], ["a", "A", "b"]).chips, ["a", "b"]);
assert.deepEqual(addLabels(["a"], ["b", "c", "d"], { max: 2 }), {
  chips: ["a", "b"],
  error: "You can add at most 2 chips",
});
assert.deepEqual(addLabels([], ["toolong", "ok"], { maxLength: 5 }).chips, ["ok"]);
console.log("chips self-check passed");
```

12. Wrap-up

Edge cases handled

- Trim, empty values, repeated separators, trailing comma.
- Case-insensitive duplicates, including duplicates inside one paste.
- Max chips and max length with a visible, announced error.
- Backspace only removes chips when the input is empty.
- Enter never submits the parent form; IME composition respected.
- Multi-line paste split correctly; blur commits pending text.

Trade-offs to mention

- **Strings vs objects:** strings are simpler and work as keys because duplicates are rejected. Allowing duplicates would need `{ id, label }`.
- **Controlled component:** the parent owns the value, so it plugs into any form library. Cost: the parent must pass `value` and `onChange`.
- **Backspace removes immediately:** some UIs (Gmail) first highlight the chip, then remove on a second Backspace. Safer against accidents, slightly more state.
- **Paste ignores caret position:** pasted text is appended to the current text. Fine for tags; a full editor would splice at `selectionStart`.
- **Blur commits:** convenient, but surprising if the user wanted to abandon the text. Could be a prop.

What to add with more time

- Unit tests for `splitLabels` / `addLabels` (Vitest) and interaction tests (React Testing Library: type, Enter, Backspace, paste).
- Autocomplete suggestions with debounced search and `aria-activedescendant`.
- Arrow-key navigation between chips, edit a chip on double-click.
- Custom `validate(label)` prop (e.g. email format) and async server validation.
- Chip variants (color, icon, avatar) via a `renderChip` prop.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why did you choose Enter and comma, but not space, as separators?

Tags often contain spaces ("Machine Learning"), so space would break them. Enter and comma are the common convention. I keep the separator as one regex (`SEPARATOR`), so adding space or `;` is a one-line change or a prop.

Q2. What if the product says duplicates are allowed?

Then the label is no longer unique, so it can't be the React key. I'd store `{ id, label }` with `crypto.randomUUID()` and remove by `id`. The dedupe step in `addLabels` would be skipped.

Q3. What if chips must be valid emails (an email "To" field)?

Add a `validate(label)` prop that returns an error string or `""`. `addLabels` calls it per label, the same way it checks `maxLength`. Invalid ones are either rejected or shown as red chips, depending on the product.

State design

Q4. Why is the chips array in the parent and not inside ChipsInput?

The chips are the form's value: the parent must read them to submit or validate. Keeping them in the parent (controlled) means one source of truth. The half-typed text is pure UI, so it stays inside.

Q5. Why don't you store `isMax` or the count in state?

They are derived from `value.length` and `max`. Storing them would create two sources of truth that can go out of sync. Derive on render; it's free.

Q6. Could you support both controlled and uncontrolled usage?

Yes: accept `defaultValue`, keep internal state when `value` is `undefined`, and use `value ?? internal` everywhere. I skipped it to keep one clear data flow within the time limit.

Challenge: comma splitting

Q7. Why handle the comma in `onChange` instead of `onKeyDown`?

`onKeyDown` only sees keys. Text can arrive by paste, autocorrect, drag-drop or mobile keyboards that don't fire reliable key events. `onChange` sees the final text, so splitting there covers all of them.

Q8. What happens with `a,,b,`?

`split` gives `["a", "", "b", ""]`. The last part (`""`) stays as input text, the rest is trimmed and filtered with `filter(Boolean)`, so only `a` and `b` become chips.

Challenge: validation

Q9. Why a `Set` instead of `chips.includes()`?

`includes` is O(n) per label, so a paste of k labels is O(n·k). The Set gives O(1) lookups and, because we add to it as we go, it also catches duplicates inside the same paste, which `includes` against the old list would miss.

Q10. Why keep the text in the input when the label is rejected?

If the user typed `Reactt` with a typo and it hits maxLength or is a duplicate, clearing it would force retyping. Keeping it lets them fix it. When at least one label is added, we clear.

Challenge: keyboard

Q11. What is `isComposing` and why check it?

For Chinese/Japanese/Korean input, the IME uses Enter to confirm a word. During that, `isComposing` is true. Without the check we'd create a chip from half-composed text.

Q12. Why `preventDefault` on Enter?

An `<input>` inside a `<form>` submits the form on Enter. A chips field should add a chip instead. Without `preventDefault`, the user submits the whole form by accident.

Challenge: paste

Q13. Why not let the browser handle all pastes?

A single-line input strips newlines, so a pasted column `Go\nRust` becomes `GoRust`. We only intercept when the clipboard contains separators; otherwise the native paste (with caret position, undo) stays intact.

Implementation steps

Q14. How does removing a chip keep keyboard users in place?

After `onChange` removes it, we call `inputRef.current.focus()`. Otherwise the focused × button disappears and focus falls back to `<body>`, and a keyboard user loses their place.

Q15. How is this accessible to a screen reader?

A real `<label htmlFor>`, chips in a `<ul>` list, each × is a `<button>` with `aria-label="Remove React"`, the error has `aria-live="polite"` and is linked via `aria-describedby`, and `aria-invalid` is set when there's an error.

Performance

Q16. What if there are 10,000 chips?

That's a UX problem first: nobody reads 10,000 chips. Show the first N and a "+9,990 more" chip, or switch to a list with search. Technically, `React.memo` on `Chip` with a stable `onRemove` avoids re-rendering all chips on each keystroke, and the Set keeps dedupe O(1).

Q17. Every keystroke re-renders all chips. Is that a problem?

For tens of chips, no. Typing updates `text` in `ChipsInput`, which re-renders its children. If it showed up in the profiler, I'd memoize `Chip` and pass `removeChip` via `useCallback` with the label as an argument.

What if / trade-offs

Q18. What if each chip must be validated by an API (e.g. user exists)?

Add the chip immediately in a "pending" state, call the API, then mark it valid or invalid (optimistic UI). Debounce or batch requests on paste, and ignore stale responses with an `AbortController`.

Q19. What if we need autocomplete suggestions?

Add a listbox under the input fed by debounced search. Arrow keys move `activeIndex`, Enter picks the active suggestion instead of the raw text, and use `role="combobox"`, `aria-expanded` and `aria-activedescendant`. Selected values are filtered out of suggestions.

Q20. Why not use a library like react-select?

In an interview, the goal is to show you can build it. In production, I'd consider it: it covers a lot of accessibility and edge cases, but it's heavy and hard to style. For a simple tag input, this ~100-line component is easier to own.

Q21. Why does Backspace delete the chip right away instead of highlighting it first?

It's simpler and matches many tag inputs. The risk is accidental deletion. A two-step version adds a `pendingRemove` flag: first Backspace highlights the last chip, second removes it, any other key clears the flag.